

Unidad 1 — Introducción a JavaScript y al cliente web

En esta primera unidad damos el salto desde el HTML y el CSS (que ya conoces) hacia el tercer pilar de la web: **JavaScript**, el lenguaje que aporta *comportamiento* e interactividad a las páginas. Aprenderás qué es y dónde se ejecuta, cómo incluirlo en una página, y los fundamentos del lenguaje moderno (ECMAScript 2025/2026): variables, tipos, operadores, control de flujo, funciones y una primera toma de contacto con el DOM. Todos los ejemplos usan **sintaxis moderna** y buenas prácticas actuales.

Índice

1. ¿Qué es JavaScript?
2. El navegador y el desarrollo web cliente
3. Cómo incluir JavaScript en una página
4. Variables y ámbito
5. Tipos de datos primitivos
6. Operadores
7. Cadenas de texto (*strings*) y *template literals*
8. Estructuras de control
9. Funciones
10. Introducción a arrays y objetos
11. El DOM: manipular la página (selección, eventos y delegación)
12. Una primera mirada a la asincronía
13. Buenas prácticas y errores comunes
14. Ejercicios propuestos
15. Recursos y referencias

¿Qué es JavaScript?

JavaScript (JS) es un lenguaje de programación que se ejecuta, originalmente, en el navegador web. Mientras que HTML define la *estructura* de una página y CSS su

presentación, JavaScript define su **comportamiento**: responder a clics, validar formularios, actualizar contenido sin recargar la página, animar elementos, comunicarse con servidores, etc.

Un poco de historia

JavaScript nació en 1995 (creado por Brendan Eich en Netscape en apenas diez días). Pese al nombre, **no tiene relación con Java**: fue una decisión de marketing de la época. Para estandarizar el lenguaje entre navegadores, se creó la especificación **ECMAScript**, mantenida por la organización ECMA bajo el estándar **ECMA-262**. Desde 2015 (ES6/ES2015) se publica una versión cada año; hoy hablamos de ECMAScript 2025/2026.

📌 Nota: cuando alguien dice "ES6", "ES2020" o "ES2024" se refiere a versiones del estándar ECMAScript. "JavaScript" es la implementación práctica de ese estándar.

¿Dónde se ejecuta?

JavaScript necesita un **motor** que lo interprete y ejecute. Los más conocidos son:

- **V8** (Google), usado en Chrome y en Node.js.
- **SpiderMonkey** (Mozilla), usado en Firefox.
- **JavaScriptCore** (Apple), usado en Safari.

Gracias a **Node.js** —un entorno que empotra el motor V8 fuera del navegador— hoy JavaScript también se ejecuta en servidores, herramientas de línea de comandos y aplicaciones de escritorio. En esta asignatura nos centramos en el JavaScript que corre **en el cliente** (el navegador).

El navegador y el desarrollo web cliente

Antes de programar conviene entender el medio en el que se ejecuta nuestro código. Esta asignatura se llama *Web Development I, the Client* precisamente porque trabajamos en el **lado del cliente**: el navegador del usuario.

Cómo funciona la web: modelo cliente-servidor

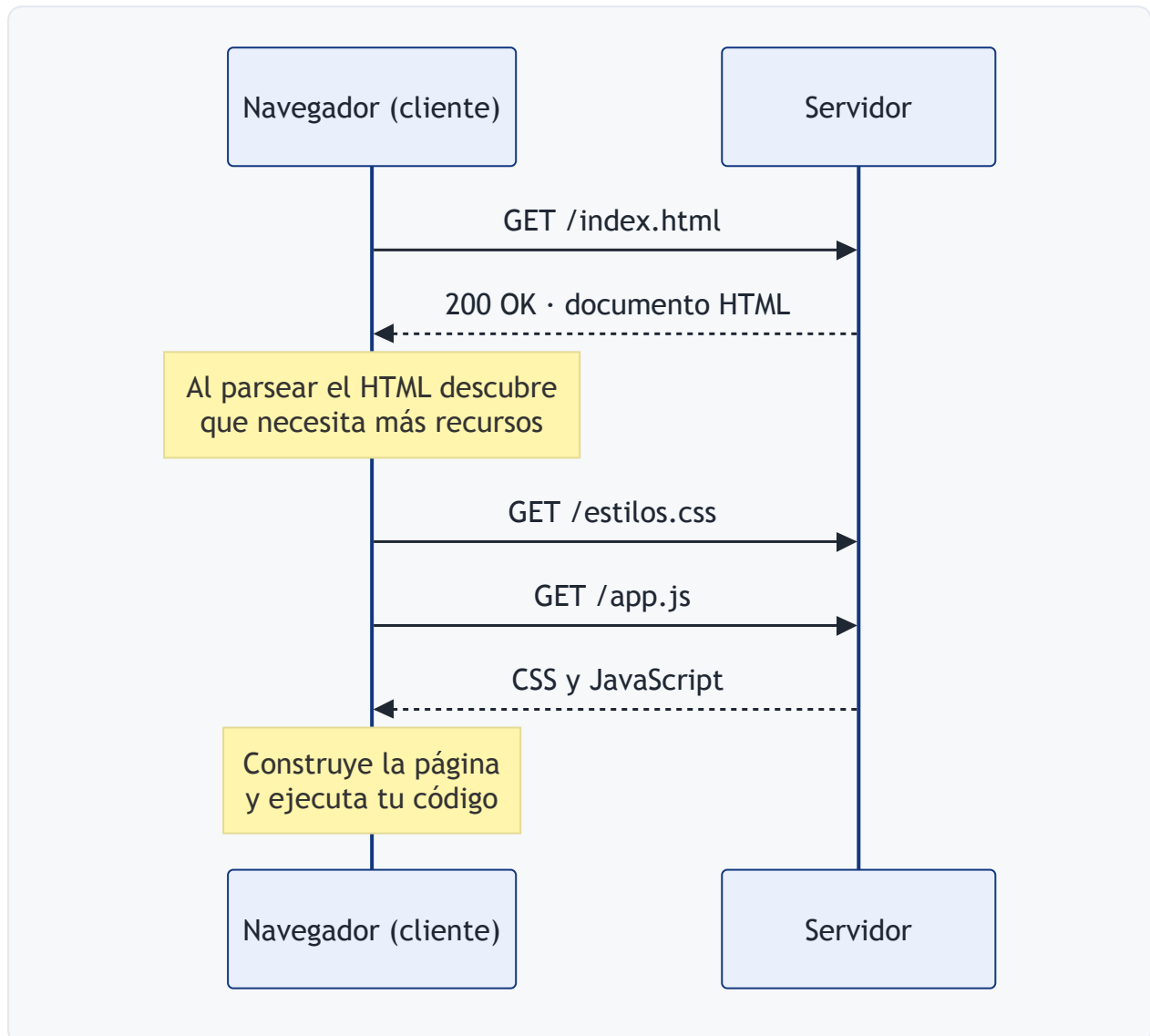
La web se basa en un diálogo entre dos partes:

- El **cliente** (el navegador) envía una **petición** (*request*) a una dirección.

- El **servidor** procesa esa petición y devuelve una **respuesta** (*response*), normalmente un documento HTML, hojas CSS, imágenes y ficheros JavaScript.

Este intercambio usa el protocolo **HTTP(S)**. Cada vez que escribes una URL o pulsas un enlace, el navegador abre una petición, recibe los recursos y construye la página.

Profundizaremos en HTTP (métodos, códigos de estado, cabeceras) en la **Unidad 4**; por ahora basta con la idea del ciclo *petición* → *respuesta*.

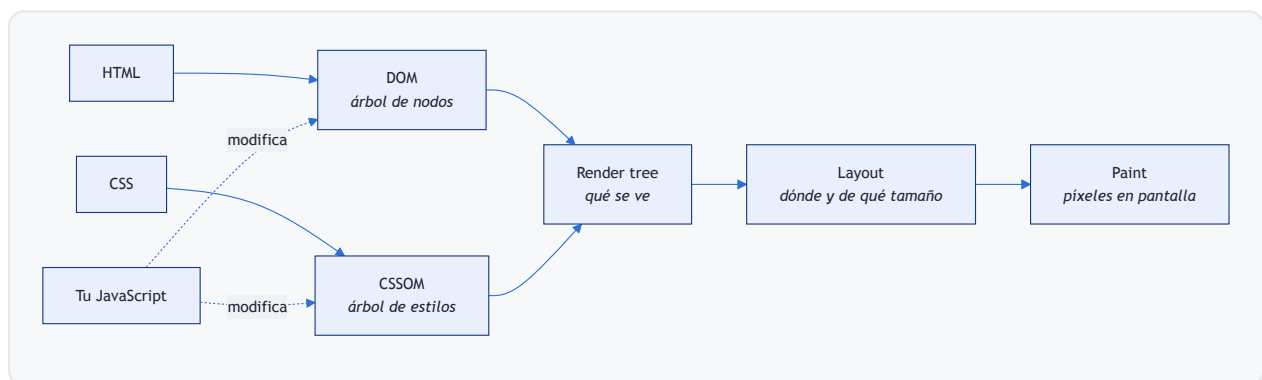


📌 Nota: el JavaScript que escribimos en esta asignatura viaja desde el servidor y se ejecuta **en el navegador del usuario**. No tiene acceso directo al disco del servidor ni a su base de datos: para eso debe pedir datos mediante peticiones HTTP (Unidad 4).

Cómo el navegador construye la página

Cuando llega el HTML, el navegador no lo muestra "tal cual": realiza un proceso por etapas (el *critical rendering path*) que conviene conocer:

1. **Parseo del HTML → DOM**: convierte el HTML en el *Document Object Model*, un árbol de objetos que representa cada etiqueta.
2. **Parseo del CSS → CSSOM**: hace lo mismo con los estilos, creando el *CSS Object Model*.
3. **Render tree**: combina DOM + CSSOM para saber qué se ve y con qué estilo.
4. **Layout (reflujo)**: calcula la posición y el tamaño de cada elemento.
5. **Paint (pintado)**: dibuja los píxeles en pantalla.



JavaScript puede intervenir en este proceso modificando el DOM y el CSSOM en vivo (lo veremos en *El DOM*). Fíjate en las flechas de puntos: cuando tu código toca el DOM, el navegador tiene que rehacer las etapas siguientes. Cambiar el DOM puede provocar un nuevo *layout* y *paint*, operaciones costosas: por eso conviene tocar el DOM con mesura.

Las tres capas del desarrollo cliente

Una página web bien construida separa **tres responsabilidades**:

Capa	Tecnología	Responsabilidad
Estructura	HTML	El contenido y su significado (semántica)
Presentación	CSS	El aspecto visual (colores, maquetación)
Comportamiento	JavaScript	La interactividad y la lógica

Esta separación facilita el mantenimiento y habilita la **mejora progresiva** (*progressive enhancement*): la página debe ser usable con HTML/CSS, y JavaScript **añade** capacidades encima, sin que su ausencia rompa lo esencial.

💡 Consejo: aunque HTML y CSS son prerequisite de esta asignatura, recuerda usar **HTML semántico** (`<header>` , `<nav>` , `<main>` , `<button>` , `<label>` ...) en lugar de `<div>` para todo. Un HTML semántico mejora la **accesibilidad** y hace que tu JavaScript trabaje con elementos correctos (por ejemplo, un `<button>` ya es accesible por teclado; un `<div>` clicable, no).

Las herramientas de desarrollo (*DevTools*)

Todo navegador moderno incluye unas **herramientas de desarrollo** que abrirás constantemente con **F12** (o *clic derecho* → *Inspeccionar*). Son tu instrumento principal de trabajo y depuración. Pestañas clave:

Pestaña	Para qué sirve
Elements (Elementos)	Inspeccionar y editar en vivo el DOM y el CSS aplicado
Console (Consola)	Ejecutar JavaScript y ver mensajes/errores (ver siguiente sección)
Network (Red)	Ver todas las peticiones HTTP: URL, estado, tiempo, respuesta
Sources (Fuentes)	Ver el código y depurar con puntos de interrupción (<i>breakpoints</i>)
Application (Aplicación)	Inspeccionar el almacenamiento del navegador: <code>localStorage</code> , <code>cookies...</code>

💡 Consejo: aprende a usar los **breakpoints** de la pestaña *Sources* en lugar de llenar el código de `console.log` . Permiten pausar la ejecución en una línea, inspeccionar variables y avanzar paso a paso. Es la diferencia entre adivinar y ver qué hace tu programa. Profundizaremos en las Web APIs del navegador (como `localStorage`) en la Unidad 2.

Cómo incluir JavaScript en una página

Se usa la etiqueta `<script>` . Lo habitual es enlazar un fichero externo `.js` :

```
<!DOCTYPE html>
<html lang="es">
  <head>
    <meta charset="UTF-8" />
    <title>Mi página</title>
    <!-- defer espera a que el HTML esté listo antes de ejecutar -->
    <script src="app.js" defer></script>
  </head>
  <body>
    <h1 id="titulo">Hola</h1>
  </body>
</html>
```

defer y async

Si colocas el `<script>` en el `<head>` sin más, el navegador detiene la construcción de la página para descargar y ejecutar el script, lo que puede dar errores (el HTML aún no existe). Para evitarlo:

Atributo	Cuándo se ejecuta	Uso típico
(ninguno)	Inmediatamente, bloqueando el HTML	Evitar
<code>defer</code>	Tras parsear todo el HTML, en orden	Recomendado para scripts propios
<code>async</code>	En cuanto se descarga, sin orden garantizado	Scripts independientes (analítica, anuncios)
<code>type="module"</code>	Como <code>defer</code> por defecto; habilita <code>import</code> / <code>export</code>	Proyectos modernos

```
<script src="app.js" type="module"></script>
```

💡 Consejo: usa `type="module"` en proyectos nuevos. Te permite organizar el código en módulos con `import` / `export` y activa automáticamente el *strict mode*.

La consola del navegador

Tu mejor amiga para aprender y depurar. Ábrela con **F12** (o clic derecho → *Inspeccionar* → pestaña *Console*). Desde el código puedes escribir en ella:

```
console.log("Hola, mundo");           // mensaje informativo
console.warn("Cuidado con esto");     // aviso
console.error("Algo ha fallado");     // error
console.table([1, 2, 3]);             // datos en forma de tabla
```

Variables y ámbito

Una **variable** es un nombre que asociamos a un valor. En JavaScript moderno se declaran con `const` y `let` (nunca con `var`).

```
const PI = 3.14159; // no se podrá reasignar
let contador = 0;    // sí se puede reasignar
contador = contador + 1;
```

- `const`: el valor **no se puede reasignar**. Úsalo por defecto.
- `let`: el valor **sí se puede reasignar**. Úsalo solo cuando necesites cambiarlo.
- `var`: forma antigua. **Evítala**.

⚠ Cuidado: `const` impide *reasignar* la variable, pero no "congela" el contenido de objetos o arrays. `const lista = []; lista.push(1);` es válido, porque no reasignas `lista`, solo modificas su contenido.

¿Por qué evitar `var`?

`var` tiene un *ámbito* (zona donde la variable existe) confuso: es de **función**, no de bloque, y al elevarse por el *hoisting* se **inicializa a `undefined`**, por lo que puede usarse antes de su declaración sin dar error (en realidad `let` y `const` también se elevan, pero permanecen inaccesibles hasta su declaración; ver la nota más abajo). `let` y `const` tienen **ámbito de bloque** (`{ ... }`), que es mucho más predecible:

```

if (true) {
  let x = 10;
  var y = 20;
}
// console.log(x); // ❌ Error: x no existe fuera del bloque
console.log("y (var):", y); // ⚠️ 20: var "escapa" del bloque (comportamiento co

// Quita el comentario de la línea de x y vuelve a ejecutar: verás el error real.

```

📌 Nota: el *hoisting* es el mecanismo por el que JavaScript "eleva" las declaraciones al inicio de su ámbito antes de ejecutar el código. Con `let` / `const` la variable existe pero no es accesible hasta su declaración (la llamada *zona muerta temporal*), lo que produce errores claros en lugar de valores `undefined` silenciosos.

Tipos de datos primitivos

JavaScript es un lenguaje de **tipado dinámico**: una variable puede contener cualquier tipo y este se determina en tiempo de ejecución. Hay **siete tipos primitivos**:

Tipo	Ejemplo	Descripción
<code>string</code>	<code>"hola"</code> , <code>'a'</code> , <code>`texto`</code>	Texto
<code>number</code>	<code>42</code> , <code>3.14</code> , <code>-7</code>	Números (enteros y decimales)
<code>bigint</code>	<code>9007199254740993n</code>	Enteros muy grandes
<code>boolean</code>	<code>true</code> , <code>false</code>	Verdadero / falso
<code>undefined</code>	<code>undefined</code>	Variable declarada sin valor
<code>null</code>	<code>null</code>	Ausencia de valor <i>intencionada</i>
<code>symbol</code>	<code>Symbol("id")</code>	Identificador único (uso avanzado)

Todo lo que no es primitivo es un **objeto** (incluidos los arrays y las funciones), que veremos más adelante.

El operador `typeof`

```
typeof "hola";    // "string"
typeof 42;        // "number"
typeof true;      // "boolean"
typeof undefined; // "undefined"
typeof null;      // "object" ⚠️ rareza histórica del lenguaje
typeof function(){}; // "function"
```

Coerción de tipos: `==` vs `===`

JavaScript a veces convierte tipos automáticamente (*coerción*). Esto provoca sorpresas:

```
console.log(''5' + 3 →, '5' + 3);    // "53" (number convertido a string → concatenación)
console.log(''5' - 3 →, '5' - 3);    // 2 (string convertido a number)
console.log('1 == "1" →, 1 == "1");  // true (== convierte tipos antes de comparar)
console.log('1 === "1" →, 1 === "1"); // false (=== compara valor Y tipo, sin coerción)

// Prueba a añadir estas joyas y explica cada una:
// console.log([] + {}, [] == false, null == undefined, null === undefined);
```

💡 Consejo: usa **siempre** `===` y `!==` (igualdad estricta). Evitan la coerción y previenen errores difíciles de detectar.

Valores *truthy* y *falsy*

Cuando un valor se usa en un contexto booleano —por ejemplo, en `if (valor)` o como operando de `||` y `&&` — JavaScript lo convierte a `true` o `false`. Los únicos valores *falsy* (los que se convierten a `false`) son:

Valor	Descripción
<code>false</code>	El propio booleano falso
<code>0</code> y <code>-0</code>	El cero numérico
<code>0n</code>	El cero de tipo <code>bigint</code>
<code>""</code>	La cadena vacía
<code>null</code>	Ausencia intencionada de valor
<code>undefined</code>	Valor sin asignar
<code>NaN</code>	Resultado numérico inválido

Todo lo demás es *truthy*, incluidos `"0"`, `"false"`, `[]` y `{}`. Por eso `if (valor)` entra en el bloque con cualquier valor *truthy*, y `valor || "por defecto"` devuelve `"por defecto"` siempre que `valor` sea *falsy*.

Conversión explícita de tipos

En lugar de depender de la coerción automática, conviene convertir los tipos de forma **explícita**:

```
Number("42");      // 42
Number("hola");    // NaN (la conversión falla)
parseInt("42px");  // 42 (lee dígitos desde el principio y descarta el resto)
String(42);        // "42"
Boolean(0);        // false
Boolean("hola");   // true
```

NaN (*Not a Number*) es el valor numérico que representa un resultado inválido: `Number("hola")` lo produce. Curiosamente, `NaN` no es igual a nada, ni siquiera a sí mismo, así que se comprueba con `Number.isNaN(valor)`.

⚠ Cuidado: el valor de un `<input>` (`input.value`) es **siempre un string**, aunque el usuario escriba un número. Conviértelo con `Number()` antes de operar con él, o `"5" + 1` te dará `"51"` en lugar de `6`.

Además, los números decimales usan el formato binario **IEEE 754**, lo que da alguna sorpresa:

```
0.1 + 0.2 === 0.3; // false (0.1 + 0.2 vale 0.30000000000000004)
```

Operadores

```
// Aritméticos
5 + 2;    // 7
5 - 2;    // 3
5 * 2;    // 10
5 / 2;    // 2.5
5 % 2;    // 1 (resto / módulo)
5 ** 2;   // 25 (exponente)

// Comparación (devuelven boolean)
5 > 3;    5 >= 5;    5 === 5;    5 !== 3;

// Lógicos
true && false; // AND → false
true || false; // OR  → true
!true;         // NOT → false

// Ternario: condición ? valorSiCerto : valorSiFalso
const edad = 20;
const mensaje = edad >= 18 ? "mayor de edad" : "menor";
```

Operadores modernos imprescindibles

```
// Nullish coalescing (??): valor por defecto solo si es null o undefined
const usuarioNombre = null;
const nombre = usuarioNombre ?? "Anónimo"; // "Anónimo"

// Optional chaining (?.): accede sin error si algo es null/undefined
const usuario = { direccion: null };
const ciudad = usuario?.direccion?.ciudad; // undefined en vez de error

// Spread (...): expande elementos
const a = [1, 2];
const b = [...a, 3, 4]; // [1, 2, 3, 4]

// Rest (...): agrupa el resto
const [primero, ...resto] = [10, 20, 30]; // primero=10, resto=[20,30]
```

⚠ Cuidado: `??` y `||` no son lo mismo. `0 || "x"` devuelve `"x"` (porque `0` es `falsy`), pero `0 ?? "x"` devuelve `0` (porque `0` no es `null` ni `undefined`). Usa `??` cuando `0`, `""` o `false` sean valores válidos.

Cadenas de texto (strings) y template literals

Las cadenas se delimitan con comillas simples, dobles o **acentos graves** (*backticks*):

```
const simple = 'hola';
const doble = "mundo";

// Template literals (backticks): interpolación y saltos de línea
const nombre = "Ana";
const saludo = `Hola, ${nombre}. Tienes ${5 * 2} mensajes.`;
const multilinea = `Primera línea
Segunda línea`;
```

Métodos útiles de *string*:

```
"Hola".length;           // 4
"Hola".toUpperCase();    // "HOLA"
" hola ".trim();          // "hola"
"hola mundo".includes("mundo"); // true
"a,b,c".split(",");       // ["a", "b", "c"]
"hola".replace("o", "0"); // "h0la" (con un string solo sustituye la PRIMERA aparición)
"coco".replaceAll("o", "0"); // "c0c0" (ES2021: sustituye todas las apariciones)
```

Estructuras de control

Condicionales

```
const nota = 7;

if (nota >= 9) {
  console.log("Sobresaliente");
} else if (nota >= 5) {
  console.log("Aprobado");
} else {
  console.log("Suspenso");
}

// switch para muchos casos discretos
const dia = "lunes";
switch (dia) {
  case "sábado":
  case "domingo":
    console.log("Fin de semana");
    break;
  default:
    console.log("Día laborable");
}
```

Bucles

```
// for clásico
for (let i = 0; i < 3; i++) {
  console.log(i); // 0, 1, 2
}

// for...of: recorre VALORES (arrays, strings...)
for (const fruta of ["manzana", "pera"]) {
  console.log(fruta);
}

// for...in: recorre CLAVES de un objeto
const persona = { nombre: "Ana", edad: 30 };
for (const clave in persona) {
  console.log(clave, persona[clave]);
}

// while
let n = 3;
while (n > 0) {
  console.log(n);
  n--;
}
```

💡 Consejo: usa `for...of` para recorrer valores de arrays y `for...in` solo para claves de objetos. Para transformar datos, los métodos de array (`map`, `filter`, `reduce`) suelen ser más claros (lo verás en la Unidad 2).

Funciones

Una función es un bloque de código reutilizable que puede recibir **parámetros** y **devolver** un valor.

Declaración vs expresión

```
// Declaración de función
function sumar(a, b) {
  return a + b;
}

// Expresión de función (guardada en una variable)
const restar = function (a, b) {
  return a - b;
};
```

Arrow functions (funciones flecha)

La forma moderna y compacta, muy usada con métodos de array y *callbacks*:

```
const sumar = (a, b) => a + b;           // return implícito
const cuadrado = x => x * x;             // un solo parámetro, sin paréntesis
const saludar = () => console.log("hola"); // sin parámetros

// Con cuerpo de varias líneas necesitas llaves y return explícito
const dividir = (a, b) => {
  if (b === 0) return null;
  return a / b;
};
```

Parámetros por defecto y rest

```
function saludarConDefecto(nombre = "invitado") {
  return `Hola, ${nombre}`;
}

saludarConDefecto();           // "Hola, invitado"
saludarConDefecto("Ana");     // "Hola, Ana"

// Rest: número variable de argumentos
function sumarTodos(...numeros) {
  return numeros.reduce((total, n) => total + n, 0);
}

sumarTodos(1, 2, 3, 4); // 10
```

Ámbito y *closures* (introducción)

Una función "recuerda" el ámbito en el que fue creada. Esto se llama **closure** (clausura) y es uno de los conceptos más potentes de JavaScript:

```
function crearContador() {  
  let cuenta = 0;           // variable "privada"  
  return () => ++cuenta;     // la función interna recuerda 'cuenta'  
}  
  
const contador = crearContador();  
contador(); // 1  
contador(); // 2
```

📌 Nota: profundizaremos en *closures*, `this` y clases en la Unidad 2. Por ahora basta con entender que una función interna puede acceder a las variables de la función que la contiene.

Introducción a arrays y objetos

Son las dos estructuras de datos fundamentales. Aquí solo lo básico para seguir los ejemplos; se estudian a fondo en la **Unidad 2**.


```
// Array: colección ordenada e indexada (desde 0)
const numeros = [10, 20, 30];
numeros[0];           // 10
numeros.length;       // 3
numeros.push(40);     // añade al final → [10, 20, 30, 40]

// Objeto: colección de pares clave-valor
const persona = {
  nombre: "Ana",
  edad: 30,
  saludar() {
    return `Hola, soy ${this.nombre}`;
  },
};
persona.nombre;        // "Ana"   (acceso con punto)
persona["edad"];       // 30      (acceso con corchetes)
persona.saludar();     // "Hola, soy Ana"
```

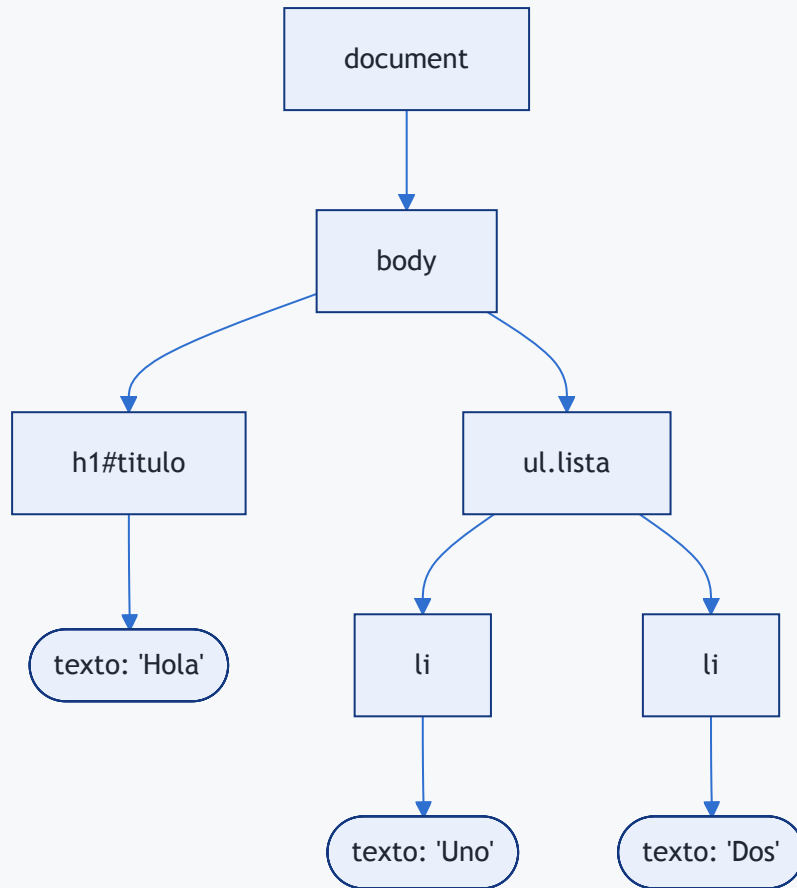
El DOM: manipular la página

El **DOM** (*Document Object Model*) es la representación en forma de árbol de objetos que el navegador hace de tu HTML. JavaScript puede leerlo y modificarlo para cambiar la página en vivo.

Este HTML...

```
<body>
  <h1 id="titulo">Hola</h1>
  <ul class="lista">
    <li>Uno</li>
    <li>Dos</li>
  </ul>
</body>
```

...se convierte en este árbol, que es lo que realmente manipulas desde JavaScript:



Dos ideas que se ven en el dibujo y que conviene tener claras desde el principio: el **texto también es un nodo** (por eso `textContent` es una propiedad y no el elemento en sí), y todo elemento tiene un **padre** y unos **hermanos**, que es justo lo que aprovecha la delegación de eventos que verás más abajo.

Seleccionar elementos

```
// El más versátil: usa selectores CSS
const titulo = document.querySelector("#titulo"); // por id
const botones = document.querySelectorAll(".boton"); // todos los .boton
const primero = document.querySelector("ul li"); // primer li dentro de ul
```

Leer y modificar contenido y estilos

```
const titulo = document.querySelector("#titulo");

titulo.textContent = "Nuevo título";    // cambia el texto
titulo.innerHTML = "<em>Hola</em>";    // cambia el HTML interno
titulo.style.color = "crimson";        // cambia un estilo en línea
titulo.classList.add("destacado");     // añade una clase CSS
titulo.classList.toggle("oculto");     // añade/quita una clase
titulo.setAttribute("data-id", "42");  // atributos
```

⚠ Cuidado: `innerHTML` interpreta el texto como HTML. Si insertas contenido proveniente del usuario sin sanear, abres la puerta a ataques **XSS**. Para texto plano usa siempre `textContent`.

Compruébalo tú: escribe `<img src=x onerror="alert('XSS')">` en la caja y mira la diferencia entre las dos formas de insertarlo.

```
<!doctype html>
<html lang="es">
<body style="font-family: system-ui; padding: 8px">
  <input id="entrada" size="42" value="&lt;b&gt;hola&lt;/b&gt;">
  <button id="pintar">Insertar</button>

  <p>Con <code>textContent</code>: <span id="seguro"></span></p>
  <p>Con <code>innerHTML</code>: <span id="peligroso"></span></p>

  <script>
    document.getElementById("pintar").addEventListener("click", () => {
      const texto = document.getElementById("entrada").value;
      document.getElementById("seguro").textContent = texto;    // se ve tal cual: <b>hola</b>
      document.getElementById("peligroso").innerHTML = texto;  // se INTERPRETA como <img src=x onerror="alert('XSS')">
    });
  </script>
</body>
</html>
```

Lo que escribes es idéntico; lo que hace el navegador con ello, no. Esa diferencia es la puerta del XSS: si ese texto viniera de un usuario, con `innerHTML` le estarías dejando

ejecutar su código en tu página.

Eventos: responder a la interacción

```
const boton = document.querySelector("#miBoton");

boton.addEventListener("click", () => {
  console.log("¡Me han pulsado!");
});

// El objeto 'event' da información del suceso
const input = document.querySelector("#nombre");
input.addEventListener("input", (event) => {
  console.log("Valor actual:", event.target.value);
});
```

Eventos habituales: `click`, `input`, `change`, `submit`, `keydown`, `mouseover`, `load`, `DOMContentLoaded`. El evento `DOMContentLoaded` (sobre `document`) se dispara cuando el HTML está completamente parseado; si cargas tu script con `defer`, ya se ejecuta justo después de ese momento y no necesitas escucharlo.

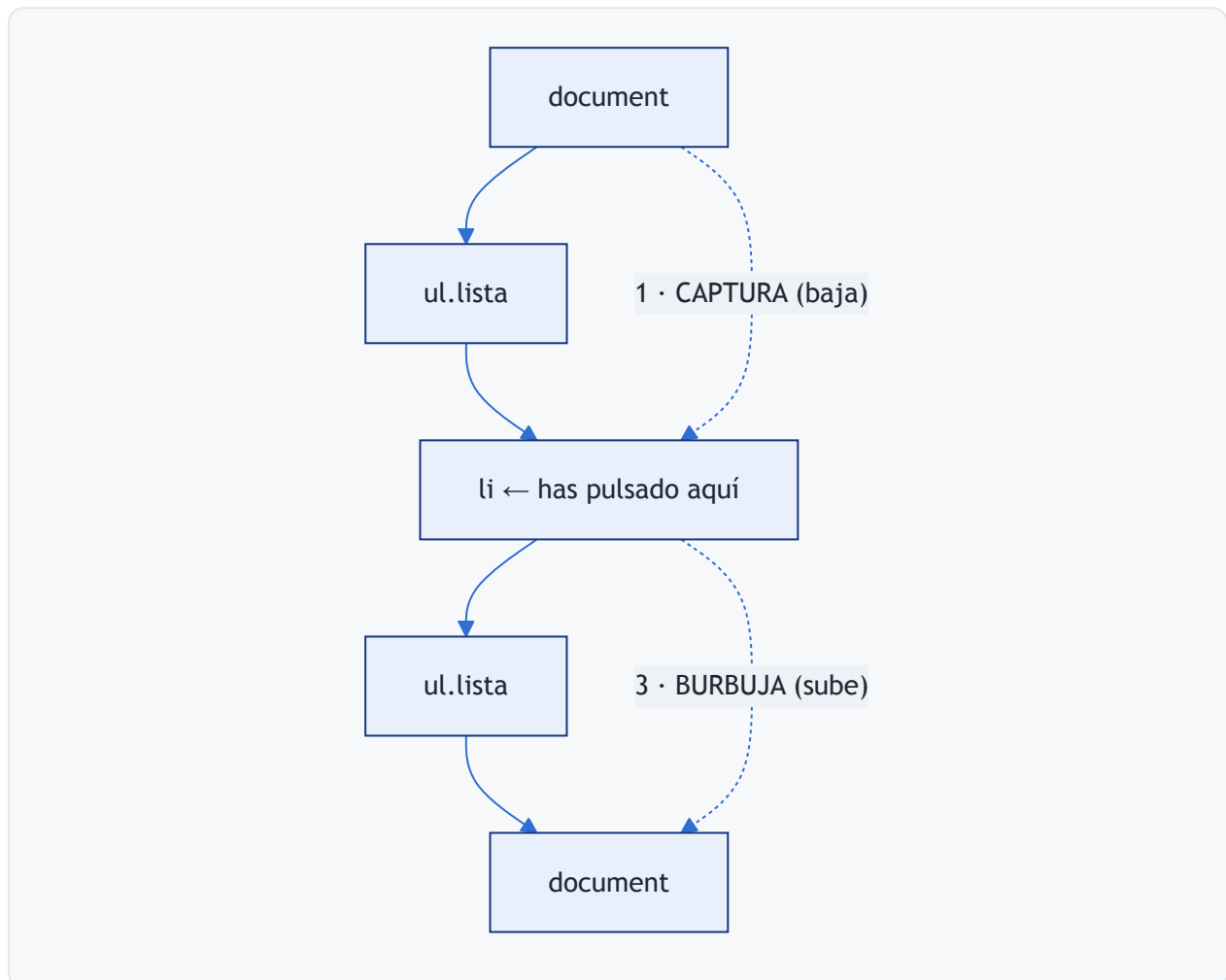
```
<!doctype html>
<html lang="es">
<body style="font-family: system-ui; padding: 8px">
  <!-- Ejemplo completo: un contador interactivo. Edítalo y vuelve a ejecutarlo. -->
  <button id="incrementar">Suma uno</button>
  <p>Llevas <strong id="valor">0</strong> clics.</p>

  <script>
    const boton = document.querySelector("#incrementar");
    const salida = document.querySelector("#valor");
    let cuenta = 0;

    boton.addEventListener("click", () => {
      cuenta++;
      salida.textContent = cuenta;
    });
  </script>
</body>
</html>
```

Propagación y delegación de eventos

Cuando pulsas un elemento, el evento no se queda en él: hace un viaje de ida y vuelta por el árbol del DOM. Primero **baja** desde `document` hasta el elemento pulsado (fase de *captura*), llega al **objetivo** (*target*) y después **sube** de vuelta (fase de *burbuja*, la que se usa casi siempre).



Que el evento **suba** tiene una consecuencia muy práctica: en lugar de poner un `addEventListener` en cada hijo, pones **uno solo en el padre** y averiguas quién lo originó con `event.target`. Eso es la **delegación de eventos**.

```

<!doctype html>
<html lang="es">
<body style="font-family: system-ui; padding: 8px">
  <ul id="lista">
    <li>Manzana</li>
    <li>Plátano</li>
    <li>Cereza</li>
  </ul>
  <button id="añadir">Añadir fruta</button>
  <p id="aviso"></p>

  <script>
    const lista = document.getElementById("lista");
    const aviso = document.getElementById("aviso");

    // UN solo listener para todos los <li>, presentes y futuros
    lista.addEventListener("click", (event) => {
      const li = event.target.closest("li");
      if (!li || !lista.contains(li)) return; // se ha pulsado fuera de un <li>
      li.style.textDecoration = li.style.textDecoration ? "" : "line-through";
      aviso.textContent = `Has pulsado: ${li.textContent}`;
    });

    // Los elementos nuevos funcionan sin registrar nada más: esa es la gracia
    document.getElementById("añadir").addEventListener("click", () => {
      const li = document.createElement("li");
      li.textContent = "Fruta " + (lista.children.length + 1);
      lista.appendChild(li);
    });
  </script>
</body>
</html>

```

💡 Consejo: la delegación es la diferencia entre un listener y doscientos. Añade elementos con el botón y comprueba que **siguen funcionando** sin haber registrado nada nuevo: con un listener por elemento tendrías que acordarte de añadirlo cada vez.

📌 Nota: `event.target` es **dónde se originó** el evento (el `<li>`, o incluso un elemento dentro de él); `event.currentTarget` es **dónde está puesto el listener** (el `<ul>`). Por eso se usa `closest("li")`: sube desde el origen hasta encontrar el `<li>` que te interesa. Si necesitas detener el viaje del evento, `event.stopPropagation()` lo corta, pero úsalo con cuidado: rompe precisamente la delegación de quien esté escuchando más arriba.

Una primera mirada a la asincronía

JavaScript es un lenguaje de **un solo hilo** (*single-threaded*): ejecuta una instrucción a la vez. Para no "congelar" la página mientras espera operaciones lentas (descargar datos, un temporizador, leer un fichero), usa un modelo **asíncrono**: encarga la tarea y sigue trabajando; cuando la tarea termina, ejecuta una función de respuesta (*callback*).

```
console.log("1");

setTimeout(() => {
  console.log("3"); // se ejecuta DESPUÉS, aunque pongamos 0 ms
}, 0);

console.log("2");

// Salida: 1, 2, 3
```

📌 Nota: este es solo el primer contacto. En la **Unidad 2** profundizaremos en el *event loop*, las **Promesas** y `async/await`, que es la forma moderna de manejar la asincronía (por ejemplo, al pedir datos a un servidor con `fetch`).

Buenas prácticas y errores comunes

- ✅ Usa `const` por defecto y `let` solo si necesitas reasignar. Olvídate de `var`.
- ✅ Compara siempre con `===` / `!==`.
- ✅ Nombra las variables de forma descriptiva (`numeroDeUsuarios`, no `n`) y en `camelCase`.
- ✅ Usa `console.log` para inspeccionar, pero retíralo del código final.

-  Activa el *strict mode* (automático con módulos `type="module"`).
-  No confundas `=` (asignación) con `===` (comparación).
-  Recuerda que los índices de array empiezan en `0`.
-  Evita variables globales: encierra el código en módulos o funciones.

Ejercicios propuestos

1. **Hola consola.** Crea un fichero `app.js`, enlázalo con `defer` en un HTML y muestra por consola tu nombre y tu edad usando un *template literal*.
2. **Tipos y `typeof`.** Declara una variable de cada tipo primitivo y muestra por consola su valor y su `typeof`. Comenta por qué `typeof null` es `"object"`.
3. **Par o impar.** Pide un número (puedes fijarlo en una variable) y muestra con un operador ternario si es par o impar.
4. **Conversor de temperatura.** Escribe una función `celsiusAFahrenheit(c)` que aplique la fórmula `c * 9/5 + 32`. Pruébala con varios valores.
5. **Clasificador de notas.** Escribe una función que reciba una nota (0–10) y devuelva `"Suspenseo"`, `"Aprobado"`, `"Notable"` o `"Sobresaliente"` usando `if/else` o `switch`.
6. **FizzBuzz.** Recorre del 1 al 30 con un bucle: imprime `"Fizz"` si el número es múltiplo de 3, `"Buzz"` si lo es de 5, `"FizzBuzz"` si lo es de ambos, y el número en otro caso.
7. **Saludo configurable.** Crea una arrow function `saludar(nombre = "invitado")` con parámetro por defecto y pruébala con y sin argumento.
8. **Contador con closure.** Implementa `crearContador()` que devuelva una función que incremente y devuelva un valor privado. Crea dos contadores independientes y comprueba que no comparten estado.
9. **DOM: cambiar texto.** Dado un `<h1 id="titulo">` y un `<button>`, haz que al pulsar el botón cambie el texto del título y le añada una clase CSS.
10. **DOM: contador interactivo.** Construye una página con un número y dos botones (`+` y `-`) que incrementen y decrementen el valor mostrado en pantalla.
11. **(Reto) Validación simple.** Dado un `<input>` de email, escribe un manejador del evento `input` que muestre en un `<span>` si el texto contiene una `@` y un punto.
12. **Explora las DevTools.** Abre cualquier web con **F12**. En la pestaña *Elements*, cambia en vivo el texto de un titular. En *Network*, recarga la página e identifica la petición del documento HTML: anota su **código de estado** y su tiempo de carga.
13. **Las tres capas.** Describe, para un botón "Me gusta" de una red social, qué parte corresponde a HTML, cuál a CSS y cuál a JavaScript. Explica qué seguiría funcionando si

JavaScript estuviera desactivado (mejora progresiva).

Recursos y referencias

- **MDN Web Docs — JavaScript:** <https://developer.mozilla.org/es/docs/Web/JavaScript>
(referencia oficial, en español)
- **MDN — Guía: Primeros pasos en JavaScript:**
https://developer.mozilla.org/es/docs/Learn_web_development/Core/Scripting
- **Eloquent JavaScript** (libro gratuito, Marijn Haverbeke): <https://eloquentjavascript.net/>
- **javascript.info — The Modern JavaScript Tutorial:** <https://javascript.info/>
- **Especificación ECMA-262 (ECMAScript):** <https://ecma-international.org/publications-and-standards/standards/ecma-262/>